🟧 Golang & Backend Fundamentals

1. Q: What Go packages did you frequently use for building RESTful APIs at Eurobank?
A: I primarily used net/http for request handling, database/sql for relational database operations, and encoding/json for marshaling and unmarshaling data. These packages formed the backbone of our API services. Together, they enabled performant and reliable endpoints for internal web and mobile applications.

2. Q: How did you ensure SQL queries were optimized in Eurobank's backend systems?
A: I normalized schemas to reduce redundancy and applied indexes on frequently queried fields. I also rewrote queries to minimize joins and leveraged prepared statements for efficiency. These optimizations reduced latency and improved response times across high-traffic services.

3. Q: Can you explain how goroutines and channels improved concurrency in your prototype services?
A: Goroutines allowed lightweight parallel execution without blocking the main thread. Channels provided a safe way to communicate between concurrent tasks, ensuring data consistency. This combination improved scalability and responsiveness under heavy load.

4. Q: What testing frameworks did you use in Go?
A: I used Go's built-in testing package for unit tests and testify for assertions and mocking. This setup allowed me to validate business logic thoroughly. It ensured reliability and reduced regressions during production deployments.

5. Q: How did you handle requirement analysis and system design in a regulated financial environment?
A: I collaborated closely with stakeholders to gather precise requirements and documented them for compliance. System designs were reviewed against regulatory standards before implementation. This ensured both technical soundness and adherence to financial regulations.

🟩 Full-Stack Development (Swaggie)

6. Q: Describe the high-throughput microservice you built at Swaggie.
A: I developed a Go microservice capable of ingesting millions of MySQL records efficiently. It used batch processing, connection pooling, and memory-optimized data handling. This design significantly improved throughput and reduced system bottlenecks.

7. Q: Which design patterns did you apply in API development?
A: I applied the singleton pattern for shared resources, the decorator for extending functionality, and the observer for event-driven workflows. These patterns improved modularity and maintainability. They also made the APIs easier to extend without rewriting core logic.

8. Q: How did you manage concurrency control in Go?
A: I used sync/atomic for lightweight atomic operations and mutexes for critical sections. Channels were employed to synchronize workloads across goroutines. This ensured safe parallel execution without race conditions.

9. Q: What containerization tools did you use?
A: I authored Dockerfiles to build lightweight images and deployed them via Kubernetes. Helm charts were used for configuration management and resource allocation. This setup provided scalability and resilience in production environments.

10. Q: How did you optimize React.js components?
A: I leveraged hooks and context APIs to reduce unnecessary re-renders. Dynamic imports helped split bundles and improve load times. These optimizations made the frontend more responsive and user-friendly.

🟧 Distributed Systems & Event-Driven Architecture (Pulsepoint)

11. Q: What Go frameworks did you use for backend services?
A: I worked with Martini, Gin-Gonic, and Beego depending on project requirements. Each framework offered different strengths, from lightweight routing to full-featured MVC support. This flexibility allowed me to tailor solutions to specific application needs.

12. Q: How did you integrate cross-language APIs?
A: I built RESTful APIs in FastAPI (Python) and Node.js alongside Go services. These APIs were designed for consistency and interoperability. This approach enabled seamless integration across multiple platforms.

13. Q: Explain your event-driven microservices architecture.
A: I used Kafka and NATS for messaging, protobuf for RPC, and service discovery for scaling. This allowed asynchronous workflows and high-throughput data pipelines. The architecture improved resilience and scalability across distributed systems.

14. Q: How did you implement observability?
A: I used logrus for structured logging and go-metrics for monitoring. These tools provided visibility into system performance and errors. They helped us quickly identify and resolve issues in production.

15. Q: What middleware did you build?
A: I implemented middleware for tracing, rate limiting, and input validation. These ensured system reliability and protected against misuse. They also improved debugging and performance monitoring.

16. Q: How did you use cobra in your projects?
A: I built CLI tools with cobra to automate internal workflows. These tools simplified repetitive tasks and improved developer productivity. They became essential for managing deployments and configurations.

17. Q: How did you ensure frontend-backend integration?
A: I developed reusable React.js libraries and connected them with APIs using Redux and React Router. This ensured consistent state management and smooth navigation. It provided a seamless user experience across applications.

18. Q: What was your leadership role at Pulsepoint?
A: I led a team of four engineers, gathering requirements and delivering production-grade systems. I coordinated tasks, reviewed code, and mentored junior developers. This role strengthened my leadership and collaboration skills.

🟥 Blockchain & Smart Contracts (VenueLog)

19. Q: What Go frameworks did you use for APIs at VenueLog?
A: I primarily used Echo and Gin-Gonic for building RESTful APIs. These frameworks provided fast routing and middleware support. They were well-suited for high-performance blockchain services.

20. Q: How did you upgrade services from HTTP/1 to HTTP/2?
A: I integrated gRPC and custom RPC layers to improve scalability. This reduced latency and supported load-balancing strategies. It allowed services to handle high-traffic loads more efficiently.

21. Q: What databases did you manage?
A: I worked with AWS S3, MongoDB, and SQL databases using pq, sqlx, and GORM. Each was chosen based on workload requirements. I implemented efficient data pipelines and storage abstractions across them.

22. Q: Which smart contracts did you develop?
A: I built ERC20 (swETH) and ERC721 (swNFT) contracts across multiple networks. I also optimized ERC1155 contracts for gas efficiency. These contracts supported staking and tokenization features.

23. Q: How did you optimize ERC1155 contracts?
A: I minimized storage layout and opcode usage to reduce gas costs. This improved transaction efficiency and scalability. It ensured cost-effective deployment on Ethereum.

24. Q: How did you connect smart contracts to networks?
A: I used Infura and Alchemy nodes for reliable connectivity. These services provided stable access to Ethereum networks. They ensured smooth contract deployment and interaction.

25. Q: How did you integrate blockchain with backend services?
A: I built Go microservices that exposed REST and gRPC interfaces. These services connected smart contracts with internal systems. This integration enabled seamless blockchain operations.

🟪 AI & LLM Integration (VenueLog)

26. Q: How did you integrate LLM APIs into VenueLog services?
A: I built a Go microservice that connected to OpenAI and Ollama APIs. It supported both REST and gRPC interfaces. This allowed flexible integration into internal tooling.

27. Q: What is Langchaingo and how did you use it?
A: Langchaingo is a Go library for prompt chaining and embedding workflows. I used it to structure AI interactions and manage context. It improved the accuracy of semantic search tasks.

28. Q: How did you implement RAG (retrieval-augmented generation)?
A: I connected Langchaingo to Qdrant for vector-based semantic search. This allowed retrieval of relevant documents before generating responses. It improved the quality and relevance of AI outputs.

29. Q: What challenges did you face integrating AI into production systems?
A: Latency and scalability were major challenges. I had to balance performance with privacy compliance. Careful architecture ensured reliable AI-powered automation.

🟧 System Design & Scalability

30. Q: How do you approach designing scalable APIs?
A: I modularize services, apply load balancing, and optimize database queries. I also use caching and versioning strategies. This ensures APIs remain performant under growth.

31. Q: How do you ensure concurrency safety in Go?
A: I use channels, mutexes, and atomic operations. These prevent race conditions and ensure safe parallel execution. It's critical for high-load systems.

32. Q: How do you handle high-throughput data pipelines?
A: I use batch processing, message queues, and efficient memory management. This ensures smooth handling of large datasets. It keeps pipelines resilient and scalable.

33. Q: How do you approach database performance tuning?
A: I apply indexing, query optimization, and caching. Schema normalization also helps reduce redundancy. These steps improve query speed and reliability.

34. Q: How do you ensure observability in distributed systems?
A: I implement logging, metrics, and tracing. Dashboards provide real-time visibility. This helps quickly identify and resolve issues.

🟫 DevOps & CI/CD

35. Q: How did you implement CI/CD pipelines?
A: I used Docker, Kubernetes, and Helm for deployments. Automated tests ensured coverage and reliability. This streamlined delivery and reduced downtime.

36. Q: How do you configure Kubernetes health probes?
A: I configure liveness probes to check if the application is running and readiness probes to verify if it can serve traffic. These probes are defined in deployment manifests and often use HTTP endpoints or command checks. Proper configuration ensures that unhealthy pods are restarted and traffic is routed only to ready instances.

37. Q: How do you manage resource limits in Kubernetes?
A: I set CPU and memory requests to guarantee minimum resources and define limits to prevent overuse. This ensures fair allocation across services and avoids resource contention. It also helps maintain cluster stability under varying workloads.

🟩 Leadership & Collaboration

38. Q: How did you mentor engineers at VenueLog?
A: I guided two frontend engineers on staking and swap page development. My focus was on backend integration, performance optimization, and explaining blockchain concepts. This mentorship ensured smooth collaboration and improved their confidence in handling complex systems.

39. Q: How do you gather client requirements effectively?
A: I conduct workshops, ask clarifying questions, and document requirements thoroughly. Iterative feedback loops help refine expectations and reduce misunderstandings. This process ensures that technical solutions align with business goals.

40. Q: How do you balance backend capabilities with frontend needs?
A: I design APIs that match UI workflows and provide consistent data structures. Performance considerations are factored in to avoid slow user experiences. This balance ensures both technical efficiency and user satisfaction.

🟦 Behavioral & Problem-Solving

41. Q: Describe a time you improved system latency.
A: At Eurobank, I optimized SQL queries and reduced response times across APIs. By indexing critical fields and rewriting queries, latency dropped significantly. This improvement enhanced user experience and reduced server load.

42. Q: How do you handle production incidents?
A: I start by triaging logs and metrics to identify root causes. If necessary, I roll back deployments to restore stability. Once resolved, I apply hotfixes and document lessons learned to prevent recurrence.

43. Q: How do you evaluate architectural improvements?
A: I build prototypes to test new approaches under simulated load. Metrics such as latency, throughput, and scalability guide decisions. This ensures that improvements are practical and beneficial before full adoption.

44. Q: How do you ensure cross-team collaboration?
A: I maintain clear documentation and schedule regular syncs with DevOps, QA, and product teams. Open communication channels help resolve blockers quickly. This collaborative approach keeps projects aligned and efficient.

45. Q: How do you prioritize tasks in a fast-paced environment?
A: I assess business impact, technical debt, and deadlines to rank priorities. Critical issues are addressed first, while long-term improvements are scheduled strategically. This balance ensures both stability and progress.

🟧 Advanced Technical Questions

46. Q: How do you implement rate limiting in Go?
A: I use middleware with token bucket or leaky bucket algorithms. This controls request flow and prevents abuse. It ensures fair usage while maintaining system performance.

47. Q: How do you handle structured logging?
A: I use logrus to log events with contextual fields. Structured logs make it easier to trace issues and analyze system behavior. They integrate well with monitoring tools for observability.

48. Q: How do you design APIs for cross-platform integration?
A: I follow REST standards, apply consistent error handling, and version APIs for backward compatibility. This ensures smooth integration across different platforms. It reduces friction for frontend and third-party developers.

49. Q: How do you ensure secure authentication in APIs?
A: I implement JWTs and OAuth2 for token-based authentication. TLS encryption secures communication channels. These measures protect sensitive data and maintain compliance.

50. Q: How do you approach modularity in system design?
A: I separate concerns into microservices and reusable libraries. Each module is independently deployable and testable. This modularity improves scalability, maintainability, and team productivity.